Database Cheatsheet

Lessons: 22 — Persistence with database/sql · 59 — Production Database Patterns

Examples: 59

Covers: `database/sql`, the pool, scanning, NULLs, transactions, isolation, migrations, query patterns

Legend: `[*]` = real Go API that the lessons have not covered yet

OPENING & THE POOL

```
import _ "github.com/lib/pq"          the driver registers itself in init()
db, err := sql.Open("postgres", dsn)  does NOT connect — it builds a pool
db.PingContext(ctx)                   the first real connection; fail fast here
defer db.Close()                      closes the pool, not "the connection"
*sql.DB is a POOL                     safe for concurrent use; ONE per process
db.SetMaxOpenConns(25)                hard ceiling; 0 = unlimited (don't)
db.SetMaxIdleConns(25)                keep them warm; match MaxOpenConns
db.SetConnMaxLifetime(5*time.Minute)  recycle — survives failovers and proxies
db.SetConnMaxIdleTime(1*time.Minute)  [*] release idle connections
db.Stats()                        [*] InUse/Idle/WaitCount — the pool's health
(passing *sql.DB around is correct; never open a DB per request)
```

DRIVERS

```
github.com/lib/pq            Postgres, pure Go, placeholders $1 $2
github.com/jackc/pgx/v5      Postgres, faster; use pgx/stdlib for database/sql
github.com/go-sql-driver/mysql    MySQL, placeholders ?
modernc.org/sqlite           SQLite, pure Go, no cgo — great for tests
mattn/go-sqlite3         [*] SQLite via cgo
(placeholders are driver-specific: $1 for Postgres, ? for MySQL/SQLite)
```

QUERYING

```
row := db.QueryRowContext(ctx, "SELECT id, name FROM users WHERE id=$1", id)
err := row.Scan(&u.ID, &u.Name)       exactly one row expected
if errors.Is(err, sql.ErrNoRows)      "not found" — an expected outcome, not a 500

rows, err := db.QueryContext(ctx, "SELECT id, name FROM users")
defer rows.Close()                    ALWAYS — it releases the connection
for rows.Next() {
  var u User
  if err := rows.Scan(&u.ID, &u.Name); err != nil { return err }
  out = append(out, u)
}
if err := rows.Err(); err != nil { return err }   the loop can end on an ERROR

db.ExecContext(ctx, "UPDATE ...", args...)   -> (sql.Result, error)
res.RowsAffected()           did it actually change anything?
res.LastInsertId()           MySQL/SQLite only; Postgres uses RETURNING
rows.Columns()           [*] column names for dynamic queries
sql.Named("id", 7)       [*] named parameters, where the driver supports them
(always the ...Context variants: they carry the cancellation and the deadline)
```

NULL & OPTIONAL COLUMNS

```
sql.NullString / NullInt64 / NullFloat64 / NullBool / NullTime
  ns.Valid                   was it non-NULL?
  ns.String                  the value (zero when NULL)
sql.Null[T]              [*] Go 1.22+: the generic version
*string                      a pointer destination also works: nil means NULL
COALESCE(col, '')            or fix it in SQL and scan into a plain string
(pick one convention per project — mixed styles are a constant source of bugs)
```

PREPARED STATEMENTS

```
stmt, err := db.PrepareContext(ctx, "SELECT ... WHERE id=$1")
defer stmt.Close()
stmt.QueryRowContext(ctx, id)
(worth it for a hot statement run in a loop; database/sql already caches per
 connection, so preparing every query by hand usually buys nothing)
```

TRANSACTIONS

```
tx, err := db.BeginTx(ctx, nil)
defer tx.Rollback()          a no-op after Commit — the safety net
tx.ExecContext(ctx, ...)     every statement goes through tx, not db
tx.QueryRowContext(ctx, ...)
return tx.Commit()           the LAST thing; its error is the real one

&sql.TxOptions{Isolation: sql.LevelSerializable, ReadOnly: true}   [*]
sql.LevelReadCommitted       the usual default
sql.LevelRepeatableRead      no non-repeatable reads
sql.LevelSerializable        full isolation; expect serialization failures
retry on 40001               [*] Postgres serialization_failure -> retry the whole tx
(keep transactions SHORT; never make an HTTP call inside one)
```

MIGRATIONS

```
golang-migrate/migrate       the common CLI + library
pressly/goose                Go-based migrations too
atlas / sqlc / ent           schema-first and codegen tools
0001_create_users.up.sql     numbered pairs ...
0001_create_users.down.sql   ... with a reversible down
one change per migration     never edit a migration that has shipped
run at deploy, not at boot   two replicas booting will race
additive first               add column -> backfill -> switch code -> drop old
(the schema is versioned code; it belongs in the repo)
```

THE QUERY PATTERNS THAT MATTER

```
N+1                          one query per row in a loop — the classic killer
  fix: WHERE id = ANY($1) then group in Go, or a JOIN
batch insert                 one INSERT with many VALUES tuples, not n round trips
  or Postgres COPY for bulk loading
keyset pagination            WHERE (created_at, id) < ($1, $2) ORDER BY ... LIMIT n
  stable and O(1); OFFSET gets slower the deeper you go
upsert                       INSERT ... ON CONFLICT (col) DO UPDATE SET x = EXCLUDED.x
                             ON CONFLICT DO NOTHING to ignore duplicates
RETURNING id, created_at     get generated values back in the same round trip
soft delete                  deleted_at timestamp + a partial index; filter everywhere
optimistic locking           UPDATE ... WHERE version = $n; 0 rows affected = conflict
pessimistic locking          SELECT ... FOR UPDATE inside a transaction
EXPLAIN (ANALYZE, BUFFERS)   read it before adding an index; Seq Scan on big = trouble
index the WHERE and ORDER BY columns, in that order
```

POSTGRES-SPECIFIC TOOLS [*]

```
SELECT ... FOR UPDATE SKIP LOCKED    the queue-worker pattern
pg_advisory_lock(key)        a cross-process mutex, held by the connection
LISTEN / NOTIFY              push notifications without polling
COPY FROM STDIN              the fastest bulk load by far
jsonb columns                index with GIN; scan into []byte and unmarshal
ANY($1) with pq.Array(ids)   pass a slice as one parameter
(these are why "just use Postgres" is usually the right answer)
```

THE REPOSITORY BOUNDARY

```
type UserRepo interface { ByID(ctx, id) (*User, error); Save(ctx, *User) error }
                             defined by the CONSUMER (the service), not the DB layer
returns domain types         not *sql.Rows, not driver errors
translates errors            sql.ErrNoRows -> domain.ErrNotFound
takes ctx first              always
a fake in-memory impl        makes the service testable with no database
read/write splitting     [*] a separate *sql.DB for replicas; beware read-after-write
(SQL belongs inside the repository — nowhere else in the codebase)
```

TRAPS & MEMORIZE

```
sql.Open doesn't connect      Ping to find out the DSN is wrong
missing rows.Close()          leaks a connection until GC; the pool drains
ignoring rows.Err()           you silently truncate the result set
ErrNoRows treated as an error the "user not found" 500
fmt.Sprintf into SQL          injection; use placeholders, always
one *sql.DB per request       destroys the pool's entire purpose
no ConnMaxLifetime            stale connections survive a failover and hang
MaxIdleConns < MaxOpenConns   silent churn: connections open and close constantly
long transactions             hold locks, block vacuum, exhaust the pool
HTTP calls inside a tx        the transaction lives as long as the remote service
SELECT *                      breaks the moment someone adds a column
OFFSET pagination at page 900 the database reads and throws away 9000 rows
missing ctx                   a cancelled request keeps its query running
```